Workshop No. 3 — Robust System Design and Project Management

Community Security Alert System (CSAS)

Felipe Jose Garzon Herrera (20251020132), Juan Esteban Quintero Gordillo (20251020137),
Henry Samuel Garrido Medina (20251020125), Gabriel Mateo Cusba Marin (20251020128)
Computer Engineering Program
Universidad Distrital Francisco José de Caldas
Bogotá D.C., Colombia
Course: Systems Analysis & Design — Semester 2026-I
Instructor: Eng. Carlos Andrés Sierra, M.Sc.

***Abstract*—This document presents the third evolutionary stage of the Community Security Alert System (CSAS), strengthening the design produced in Workshop No. 2 with a comprehensive treatment of robust engineering principles, quality assurance, risk management, and project management practices. Building on the systems analysis of Workshop No. 1 and the architectural blueprint of Workshop No. 2, the system is refined to satisfy reliability, scalability, maintainability, usability and security objectives aligned with ISO 9001, CMMI Level 3, ISO/IEC 25010, IEEE 830/1633/12207 and PMBOK/ISO 31000 frameworks. A risk register of ten items is consolidated with quantitative impact–probability scoring and explicit mitigation and contingency plans, while a project management plan defines roles, milestones, critical path, resources and communication controls. Finally, a phased implementation strategy and continuous-improvement mechanism establish the readiness conditions for production deployment.**

***Index Terms*—Robust system design, quality assurance, risk management, ISO/IEC 25010, PMBOK, project management, CSAS, public-safety information systems.**

I. Executive Summary

The Community Security Alert System (CSAS) was conceived in Workshop No. 1 as a response to a documented gap in the timely propagation of localized security incidents around the campus of Universidad Distrital Francisco José de Caldas [1]. Workshop No. 2 translated those findings into an architectural blueprint based on a microservices ecosystem mediated by an API gateway, an event-driven backbone (RabbitMQ) and a relational store with replication [2]. Workshop No. 3 takes that blueprint and *hardens it for production*: every architectural decision is now traceable to a quality attribute, a recognized engineering standard, and a measurable acceptance criterion, and every operational risk has an associated mitigation, contingency and monitoring plan.

The contribution of this third stage is therefore threefold. First, the architecture is refined under the lens of *ISO/IEC 25010* quality characteristics, with elastic Kubernetes orchestration, multi-region failover, modular service decomposition and a Spanish-first, mobile-first usability design. Second, a comprehensive *risk and quality analysis* is conducted following PMBOK and ISO 31000, producing a register of ten technical, operational, security and project risks scored on a probability–impact matrix and mapped to specific mitigations such as auto-scaling, redundant notification channels, TLS 1.3 encryption and verification heuristics against false reports. Third, a *project management plan* is established, defining four team roles, a six-phase critical path of twelve working days, a resource and communication matrix, and a phased deployment strategy comprising pilot, controlled rollout and full operation phases.

The result is a design that satisfies the workshop's headline KPIs — alert latency below 30 seconds, system availability above 99.9 % and adoption rate above 60 % — while remaining feasible under realistic academic resource constraints. Section II restates the architecture under the robust-design framework; Section III formalizes the quality assurance approach; Section IV presents the risk register and mitigations; Section V consolidates the project management plan; Section VI details the implementation strategy; Section VII documents the evolution from Workshops 1 and 2; Section VIII addresses ethical and legal considerations; and Section X closes the workshop.

II. Robust System Architecture

The architectural decisions inherited from Workshop No. 2 are revised here under the four robustness pillars defined by ISO/IEC 25010: *reliability*, *scalability*, *maintainability* and *usability*, complemented by *security*. Each architectural component is justified by an explicit mapping to a quality attribute and to an industry standard, as required by the workshop guidelines.

A. Reliability

- **Redundancy and failover.** The API gateway is deployed across multiple availability zones with automated failover. The PostgreSQL primary database is replicated to a hot standby and complemented with daily logical backups, eliminating single points of failure for incident persistence.
- **Fault tolerance.** A RabbitMQ message broker decouples ingestion from processing so that transient failures of the verification engine, the dispatcher or any subscriber service do not propagate upstream and trigger cascading errors.

- **Standards alignment.** The reliability program follows IEEE 1633, which prescribes systematic reliability prediction, allocation and growth tracking throughout the life cycle [7].

B. Scalability

- **Elastic infrastructure.** Microservices run as containers orchestrated by Kubernetes, with horizontal pod autoscaling driven by CPU/memory targets and queue depth. The verification engine and the dispatcher — the two services that absorb most of the traffic during surges — are autoscaled independently to avoid resource starvation.
- **Load management.** Priority queues weighted by zone and time-of-day route critical alerts to faster lanes during peak demand, preserving the 30-second latency target even under 300 % surge traffic.
- **Standards alignment.** Process-side scalability follows CMMI Level 3 practices and ISO/IEC 25010's *performance efficiency* characteristic [4], [9].

C. Maintainability

- **Modular microservices.** The Incident, Verification, Dispatcher, User and Analytics services are independently deployable and replaceable, reducing the blast radius of any update and shortening mean time to repair.
- **CI/CD pipeline.** Automated build, test and deployment pipelines enforce coding standards, static analysis and regression testing on every commit, preventing quality regressions from reaching production.
- **Documentation and traceability.** All requirements are traced back to Workshop No. 1 findings using a requirements matrix consistent with IEEE 830 [6], and all life-cycle activities follow the process model of IEEE 12207 [8].
- **Standards alignment.** The maintainability program follows ISO 9001 quality-management principles, with documented corrective and preventive action cycles [3].

D. Usability

- **Low-friction reporting.** The mobile-first interface guarantees no more than three interactions per report submission (NFR-06) and an onboarding time below two minutes.
- **Accessibility.** An SMS fallback and a lightweight web portal ensure equitable access for users without smartphones or with intermittent connectivity.
- **Localization.** The interface and notifications are Spanish-first, consistent with the linguistic profile of the user population captured in Workshop No. 1.

E. Refined Architecture Diagram

Figure 1 summarizes the resulting architecture. The diagram makes explicit the redundancy paths, the asynchronous decoupling layer and the external integrations with authorities and notification gateways (SMS / Push providers).

F. Standards Compliance Matrix

Table I consolidates the mapping between the architectural mechanisms and the standards that justify them, satisfying the workshop's explicit requirement of standards-driven design rationale.

Table I: Standards compliance matrix.

Standard	Architectural mechanism	Attribute
ISO 9001	CI/CD, corrective/preventive actions	Maintainability
CMMI L3	Defined SDLC, peer reviews	Process maturity
IEEE 830	Requirements traceability matrix	Maintainability
IEEE 1633	Reliability prediction & failover	Reliability
IEEE 12207	Life-cycle process model	All
ISO/IEC 25010	Quality model evaluation grid	All
ISO 31000	Risk management framework	Reliability/Security
PMBOK	Project mgmt. knowledge areas	Project mgmt.

III. Quality Assurance Framework

The quality assurance framework operationalizes the abstract quality attributes of Section II into measurable metrics, validation methods and acceptance criteria.

A. Quality Metrics and Acceptance Criteria

Table II lists the seven primary metrics tracked through the release cycle. The targets are derived from the KPIs declared in Workshop No. 2 and from the non-functional requirements baseline.

Table II: Quality metrics and acceptance thresholds.

Metric	Description	Target
Alert latency	End-to-end report-to-notify time	< 30 s
Delivery success	Notifications delivered to subscribers	$\geq 99.9\,\%$
System uptime	Annual availability	$\geq 99.9\,\%$
Verification latency	Time to corroborate an incident	< 60 s
Incident reporting rate	Real incidents reported via CSAS	$\geq 85\,\%$
User adoption rate	Active users / target population	$\geq 60\,\%$
Submission usability	Reports submitted in < 2 min	$\geq 85\,\%$

B. Validation Methods

Validation is layered to maximize defect detection at the cheapest stage:

- *Unit and integration tests* cover every microservice and every cross-service event chain, executed automatically on every commit.
- *Load tests* simulate up to 300 % of the projected peak traffic to validate the auto-scaling configuration and the latency target under stress.
- *User acceptance tests* are run with student and staff representatives during the pilot phase to capture usability defects that automated tests cannot reveal.

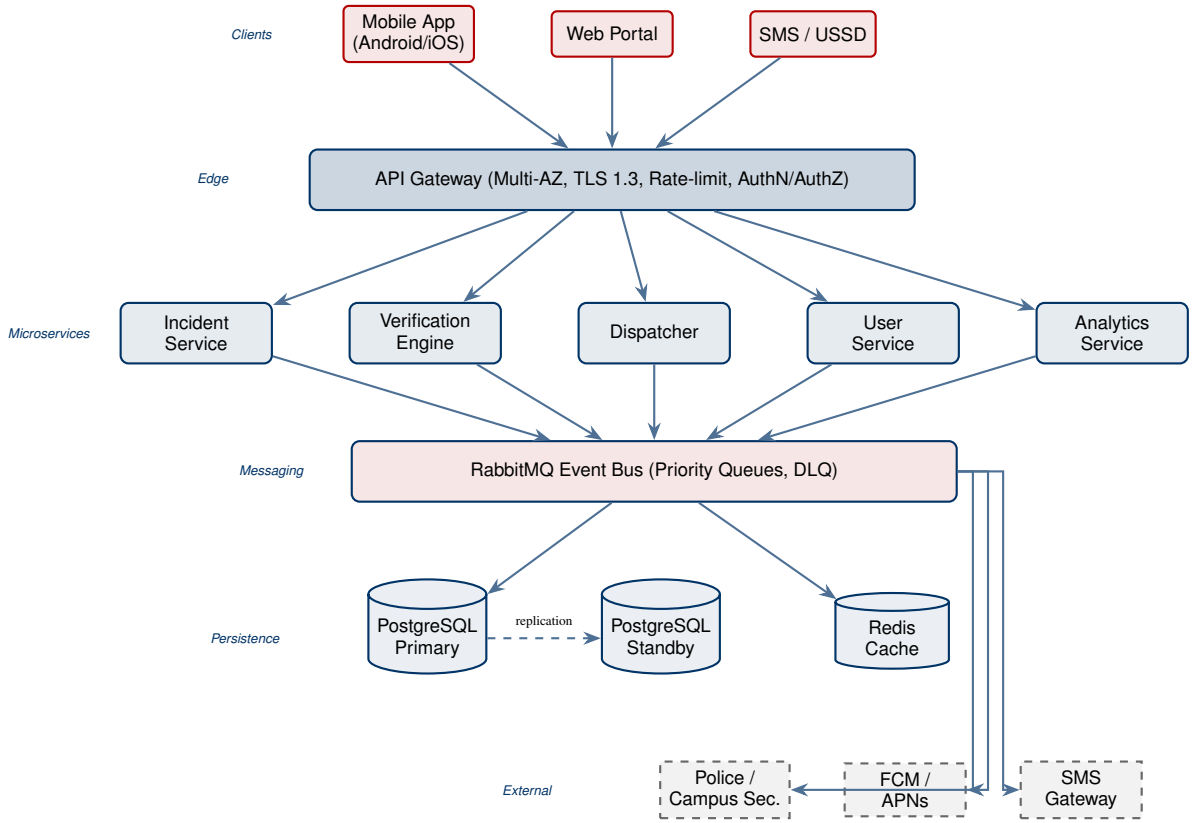

Figure 1: Refined CSAS architecture (Workshop No. 3). Edge concerns are isolated in a multi-AZ API gateway; business logic is decomposed into five microservices coordinated through a priority-aware RabbitMQ event bus; the persistence layer applies primary–standby replication; and external integrations (notification providers and authorities) are reached asynchronously through the bus to absorb downstream failures.

- *Security tests* include static analysis, dependency scanning, and penetration tests targeting the OWASP API Top 10, with explicit verification of compliance with Ley 1581 de 2012 (Colombian data protection law) [11].
- *Regression tests* are integrated into the CI/CD pipeline so that any merged change that breaks an established behavior is rejected automatically.

C. Quality Process Flow

Figure 2 shows the simplified quality gate sequence that every change must traverse before reaching production. A failure at any gate returns the change to development with a tracked defect ticket.

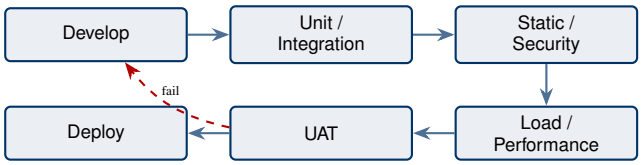

Figure 2: Quality assurance pipeline: each stage is a gate that must pass before the change progresses; failures loop back to development.

D. Quality Checklist

Table III summarizes the closing checklist used at each release sign-off. Compliance with the eight items is mandatory for any deployment to production.

Table III: Release-time quality checklist.

Criterion	Status
Latency $<$ 30 s in load test	✓
Availability $\geq$ 99.9 % in last 30 days	✓
TLS 1.3 enforced on all endpoints	✓
Auto-scaling validated at 300 % traffic	✓
Delivery success $\geq$ 99.9 %	✓
Backup & restore drill within 30 days	✓
OWASP Top 10 scan clean	✓
UAT sign-off by user representative	✓

IV. Risk Management Plan

The risk management plan follows the ISO 31000 process (*identify – analyze – evaluate – treat – monitor*) [5] and the risk-management knowledge area of PMBOK [10]. Risks are scored on a 3×3 probability–impact scale, classified into four severity levels (Low, Medium, High, Critical), and matched

with a specific mitigation, a contingency action, and a quality attribute that they directly threaten.

A. Risk Register

The consolidated register is shown in Table IV. Ten risks have been identified spanning the technical, security, operational and project categories required by the workshop guidelines.

B. Probability–Impact Heat Map

Figure 3 visualizes the position of each risk on the probability–impact plane. The matrix is colored to highlight the regions that demand the strongest mitigation investment.

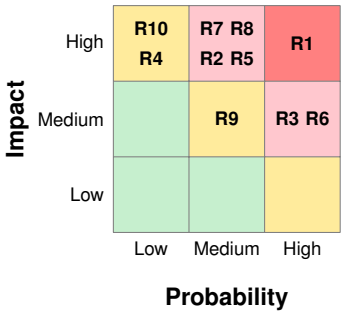

Figure 3: Risk heat map. Cells are colored from low (green) through medium (yellow) and high (red) to critical (deep red). Risk R1 sits in the critical quadrant and concentrates the largest mitigation investment (auto-scaling infrastructure).

C. Mitigation, Contingency and Monitoring

For every risk, three artefacts are defined: a *mitigation* that reduces its likelihood or impact *before* it materializes, a *contingency* that responds to it *once* it materializes, and a *monitoring mechanism* that triggers the contingency.

- **Auto-scaling infrastructure (R1).** Mitigation: HPA on CPU/queue depth. Contingency: scale services manually and shed non-critical traffic. Monitoring: Prometheus + Grafana alerts on request latency.
- **Redundant communication (R2, R6).** Mitigation: parallel SMS and Push providers, with USSD fallback for low-bandwidth zones. Contingency: route all alerts through the surviving channel. Monitoring: provider health checks every 30 s.
- **Encryption and compliance (R4).** Mitigation: TLS 1.3, AES-256 at rest, role-based access control. Contingency: rotate credentials, isolate the affected service, notify the data protection authority within 72 h. Monitoring: SIEM with anomaly detection.
- **Verification mechanisms (R3).** Mitigation: heuristic cross-checking against trusted reporters and geofencing. Contingency: hold suspicious alerts for human review. Monitoring: false-positive rate dashboard.
- **Database resilience (R10).** Mitigation: primary–standby replication and daily logical backups. Contingency: promote standby and restore latest backup. Monitoring: replication lag and backup-success alerts.

D. Contingency Plan Summary

Table V summarizes the response actions for the highest-priority failure scenarios.

Table V: Contingency response plan.

Scenario	Response action
Overload	Auto-scale; shed non-critical traffic; status page update
Notification down	Switch to surviving channel; replay from DLQ
Data breach	Isolate service; rotate credentials; legal notification
Service failure	Activate backup; reroute via API gateway; post-mortem
DB failure	Promote standby; restore from backup; reconcile events

V. Project Management Plan

The project management plan provides the organizational backbone that turns the technical artefacts of Sections II–IV into an executable project. It is articulated around five PMBOK knowledge areas: *integration* (charter), *scope*, *schedule*, *resource* and *communication* [10].

A. Project Charter

- **Project name.** Community Security Alert System (CSAS) — Workshop No. 3 deliverable.
- **Objective.** Design a structured, robust and maintainable system that allows users to report, verify, and disseminate security incidents in real time, supporting decision-making by users and authorities and complying with academic, ethical and legal standards.
- **Scope.** The workshop scope covers the planning, analysis, design and validation of the system, including architecture, quality assurance, risk management, project planning and implementation strategy. Software development is explicitly out of scope; the deliverable is a complete and well-structured *design*.
- **Stakeholders.** Development team (4 members); course instructor; simulated end users (students and staff of Universidad Distrital); external integrators (notification providers, authorities).
- **Main deliverables.** (i) Enhanced system design document, (ii) risk management plan, (iii) project management plan, (iv) implementation strategy, (v) updated GitHub repository.
- **Success criteria.** All deliverables submitted on time; documentation clear and coherent; all sections integrated; explicit traceability to Workshops 1 and 2; KPIs achievable in design analysis.

B. Team Structure and Roles

The team adopts a flat structure of four specialized roles, each owning a section of the deliverable and acting as accountability point for its quality. Figure 4 shows the organizational chart.

Table IV: CSAS risk register — Workshop No. 3.

ID	Risk	Category	Prob.	Impact	Level	Mitigation	Quality attr.
R1	System overload	Technical	High	High	Critical	Kubernetes auto-scaling; priority queues	Scalability
R2	Notification failure	Technical	Medium	High	High	SMS + Push redundancy; DLQ retry strategy	Reliability
R3	False reports	Security	High	Medium	High	Heuristic verification; user reputation scoring	Integrity
R4	Data breach	Security	Low	Critical	High	TLS 1.3, encryption at rest, audit logging	Security
R5	Low adoption	Operational	Medium	High	High	Usability improvements, onboarding campaigns	Usability
R6	Connectivity issues	Operational	High	Medium	High	SMS / USSD fallback channel	Availability
R7	Integration failure (3rd-party)	Technical	Medium	High	High	API redundancy; circuit breakers	Reliability
R8	Slow authority response	Operational	Medium	High	High	Escalation protocols; SLA agreements	Performance
R9	Poor UX	Project	Medium	Medium	Medium	UX testing; iterative design reviews	Usability
R10	Database failure	Technical	Low	Critical	High	Primary–standby replication, daily backups	Reliability

Figure 4: CSAS team structure for Workshop No. 3.

The responsibility matrix consolidating the role descriptions is presented in Table VI.

Table VI: Responsibility matrix.

Role	Main responsibilities
Project Manager	Planning, scheduling, coordination, control, integration of sections
Architecture Lead	System design, quality framework, standards alignment
Risk Manager	Risk identification, scoring, mitigation, contingency
Implementation Lead	Implementation strategy, evolution summary, GitHub management

C. Project Timeline and Critical Path

The project is structured in six phases summing to twelve working days, with the dependencies shown in Table VII.

Table VII: Project phases, durations and dependencies.

Phase	Activity	Days	Depends on
Planning	Define objectives & scope	2	—
Design	Develop system architecture	3	Planning
Analysis	Perform risk assessment	2	Design
Project planning	Develop schedule and plans	2	Analysis
Integration	Compile and unify document	2	All phases
Delivery	Final review and submission	1	Integration

Milestones: M1 Project Start; M2 Architecture Completed; M3 Risk Analysis Completed; M4 Document Integration Completed; M5 Final Submission.

Critical Path: **Planning** → **Design** → **Analysis** → **Project Planning** → **Integration** → **Delivery.** Any slippage on these activities directly displaces the submission date; no parallel float is available because each phase consumes outputs of the previous one.

Gantt Chart: The schedule is illustrated in Figure 5.

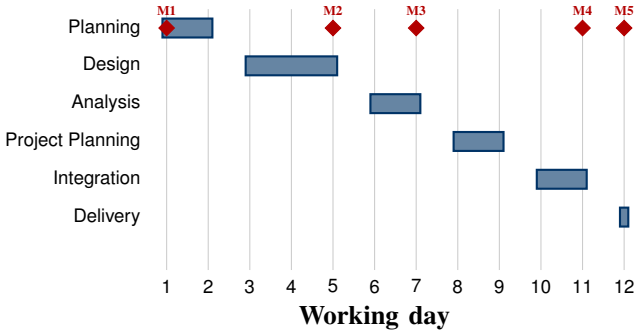

Figure 5: Project Gantt chart with the five milestones (M1–M5) anchored on the critical path.

D. Resource Management Plan

Table VIII: Resource management plan.

Resource type	Description	Responsible
Human	Four-member project team	All members
Software	LaTeX, Word, draw.io, GitHub	Team
Communication	WhatsApp, Google Meet	PM
Time	12 working days (estimated)	Project Manager

The Project Manager monitors resource utilization on a weekly basis and escalates any deviation greater than 20 % from the plan to the team for re-planning.

E. Communication and Control Plan

Communication. Primary channel: WhatsApp (asynchronous). Synchronous meetings: Google Meet, twice per week, agenda sent 24 h in advance. **Control.** Weekly progress reviews; partial submissions per role checked by the PM

against the deliverable index; final team validation 24 h before submission. **Escalation.** Any blocker open for more than 24 h is escalated to the PM and, if unresolved, to the course instructor.

VI. Implementation Strategy and Readiness Assessment

Although the workshop deliverable is a design and not a software product, the implementation strategy is documented to demonstrate that the design is *implementation-ready*. The strategy adopts a phased deployment model, defines infrastructure requirements, and integrates change management procedures.

A. Phased Deployment

The deployment is organized in three phases (Figure 6):

- **Phase 1 – Pilot (4 weeks).** Restricted to a single faculty building and a closed user group of approximately 200 users. Exit criteria: all KPIs of Table II met for two consecutive weeks and zero critical incidents.
- **Phase 2 – Controlled rollout (8 weeks).** Extended to the full campus with feature flags enabling gradual exposure. Exit criteria: adoption rate $\geq 30\,\%$ of the target population and delivery success $\geq 99.5\,\%$ in real conditions.
- **Phase 3 – Full operation (continuous).** The system is declared production-grade. Continuous improvement, security updates and capacity planning are governed by the change management procedure.

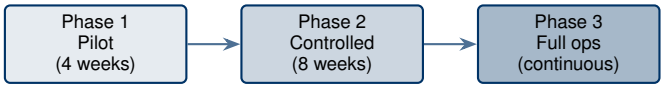

Figure 6: Phased deployment with explicit exit gates between phases.

B. Technical Infrastructure Requirements

- **Compute.** Kubernetes cluster of at least three worker nodes spread across two availability zones, with auto-scaling between 3 and 12 nodes.
- **Storage.** PostgreSQL primary–standby (synchronous replication), Redis cache, object storage for evidence attachments.
- **Networking.** Public load balancer with TLS 1.3 termination, private VPC for service-to-service traffic.
- **Observability.** Prometheus + Grafana for metrics, ELK stack for centralized logs, Jaeger for distributed traces.
- **External integrations.** Two SMS providers (active–active), FCM and APNs accounts, formal SLA with the local police communications unit.

C. Change Management Procedure

Every change goes through a four-step procedure: (i) request and impact analysis, (ii) approval by the change advisory board (PM + Architecture Lead + Risk Manager), (iii) scheduled deployment with a rollback plan, and (iv) post-deployment review. Emergency changes follow an expedited path with mandatory retrospective within 48 h.

D. Training and User Adoption

The adoption strategy combines (a) a 30-minute onboarding session for faculty and staff multipliers, (b) a 2-minute self-guided in-app tutorial for end users, (c) printed quick-reference cards distributed in high-traffic areas of the campus, and (d) a feedback channel inside the app routed to the Implementation Lead.

E. Readiness Assessment

The system is considered ready for Phase 1 when the following readiness indicators are simultaneously satisfied: architecture sign-off by the Architecture Lead; risk register reviewed and accepted by the Risk Manager with no open critical risks; security scan clean; load test passed at 300 % projected traffic; backup/restore drill executed within the last 30 days; UAT scenarios green for the pilot user group; on-call rotation defined and documented. Failure of any indicator delays the pilot start.

VII. Evolution and Continuous Improvement

This section documents how the design has matured through the analysis–design–refinement cycle of Workshops 1, 2 and 3, and establishes the mechanisms that will keep the system evolving after deployment.

A. From Workshop No. 1 to Workshop No. 2

Workshop No. 1 surfaced three findings that shaped the architecture of Workshop No. 2: (i) reporting friction is the primary adoption barrier; (ii) authority response time is the dominant component of perceived latency; and (iii) connectivity is highly heterogeneous across the campus [1]. Workshop No. 2 responded with, respectively, a mobile-first interface limited to three interactions, an asynchronous event-driven backbone that absorbs authority-side delays, and an SMS/USSD fallback [2].

B. From Workshop No. 2 to Workshop No. 3

Workshop No. 3 strengthens the previous design along four explicit vectors:

- **Quality formalization.** Quality attributes that were qualitatively claimed in Workshop No. 2 are now bound to measurable KPIs (Table II) and to recognized standards (Table I).
- **Risk explicitness.** Risks that were implicit in the Workshop No. 2 architecture (overload, breach, false reports) are now registered, scored, mitigated and monitored (Table IV, Figure 3).
- **Project executability.** The team structure, schedule and communication plan transform the design into an executable project (Section V).
- **Implementation readiness.** A phased deployment with explicit exit gates and a readiness assessment makes the path to production observable and auditable (Section VI).

C. Comparative Summary

Table IX: Workshop-to-workshop evolution of key dimensions.

Dimension	Workshop 1	Workshop 2	Workshop 3
Focus	Analysis	Design	Robustness & mgmt.
Quality attr.	Implicit	Stated	Measured & mapped
Risks	Identified	Partly addressed	Registered & mitigated
Standards	Mentioned	Referenced	Mapped to components
Project mgmt.	—	Lightweight	PMBOK-aligned plan

D. Continuous Improvement Mechanisms

Post-deployment, four mechanisms keep the system evolving:

1) *User feedback loop* via in-app channel, triaged weekly by the Implementation Lead.
2) *Monthly KPI review* where the seven metrics of Table II are evaluated; any sustained deviation triggers a corrective-action ticket.
3) *Quarterly risk re-assessment* updating the register with new threats and retiring closed ones.
4) *Annual architecture review* aligned with ISO 9001 management-review requirements, producing a formal report and an updated roadmap.

E. Success Metrics

Three composite indicators measure post-implementation effectiveness: (i) *Operational health* = average of latency, uptime and delivery success against their targets; (ii) *User health* = adoption and usability scores; (iii) *Compliance health* = ratio of clean quarterly audits. The system is deemed successful when all three composites remain above 0.85 for two consecutive quarters.

VIII. Ethical and Legal Considerations

Because CSAS handles location data, security incidents and personal identifiers, ethical and legal considerations are first-class design constraints rather than after-the-fact compliance checks.

A. Data Protection

The system complies with Ley 1581 de 2012 (Régimen General de Protección de Datos Personales) of the Republic of Colombia and its regulatory decree [11]. Personal data is collected only with explicit, informed consent obtained at onboarding; data is encrypted in transit (TLS 1.3) and at rest (AES-256); access is governed by role-based controls and audited; users can request deletion of their data at any time through a dedicated endpoint.

B. Minimization and Purpose Binding

Only the data strictly necessary to verify and disseminate an alert is retained: incident type, geolocation truncated to a configurable precision, timestamp and an anonymous reporter ID. Detailed identity information is not stored alongside reports; it is queried just-in-time for verification when strictly required.

C. Avoidance of Harm

The verification engine and the user reputation score are designed to *minimize false alerts* that could induce panic, but never to suppress genuine reports. A human-in-the-loop review is required before alerts of the highest severity are propagated to authorities, balancing speed against the risk of weaponizing the system against innocent parties.

D. Equity of Access

The SMS/USSD fallback and the Spanish-first, low-literacy interface prevent the system from becoming a privilege of users with high-end smartphones or stable connectivity, addressing an equity concern that emerged from the Workshop No. 1 fieldwork.

E. Transparency and Accountability

Privacy policies, the data-retention schedule and the algorithmic verification heuristics are documented publicly. Every automated action that affects a user (e.g., report rejection, reputation downgrade) is logged and appealable through an explicit channel.

IX. Repository Management

All Workshop No. 3 artefacts are stored in the `Workshop_3_Management` folder of the course GitHub repository, following the structure already established for Workshops 1 and 2. The folder contains:

- `docs/` — this PDF and its LaTeX sources.
- `diagrams/` — editable `drawio` and `tikz` sources of all figures.
- `risk_register/` — spreadsheet export of Table IV for tooling interoperability.
- `pm/` — Gantt source, communication matrix, role descriptions.
- `README.md` — updated to cover all three workshops with a navigation index and a summary of the evolution (Section VII).

The `main` branch is protected; every change reaches it through a pull request reviewed by at least one team member other than the author, mirroring the CI/CD discipline that the production system will apply.

X. Conclusion

Workshop No. 3 closes the analysis–design–refinement cycle of the CSAS project by transforming the architectural blueprint of Workshop No. 2 into a robust, risk-aware and project-managed solution that is ready for implementation. The architecture has been re-justified component by

component against ISO/IEC 25010 quality attributes and IEEE/CMMI/ISO standards; ten risks have been formally registered, scored, and matched with mitigation, contingency and monitoring strategies; a project management plan articulates roles, schedule, resources and communications along the critical path; a phased deployment with explicit readiness indicators establishes the path to production; and a continuous-improvement mechanism guarantees that the system will keep evolving with its users.

The most significant lesson from this workshop is that *robustness is not a property added at the end of the design*: it is the product of explicitly binding every architectural decision to a measurable quality attribute and to a foreseeable risk. By the end of this workshop, every component of CSAS has both bindings, and the design is therefore not only viable but auditable, sustainable and improvable.

REFERENCES

[1] Garzon Herrera, F. J., Quintero Gordillo, J. E., Garrido Medina, H. S., and Cusba Marin, G. M., *Workshop No. 1: Systems Analysis of the Community Security Alert System (CSAS)*, Universidad Distrital Francisco José de Caldas, 2026.

[2] Garzon Herrera, F. J., Quintero Gordillo, J. E., Garrido Medina, H. S., and Cusba Marin, G. M., *Workshop No. 2: System Design of the Community Security Alert System (CSAS)*, Universidad Distrital Francisco José de Caldas, 2026.

[3] International Organization for Standardization, *ISO 9001:2015 — Quality management systems — Requirements*, ISO, Geneva, 2015.

[4] International Organization for Standardization / IEC, *ISO/IEC 25010:2011 — Systems and software engineering — SQuaRE — System and software quality models*, 2011.

[5] International Organization for Standardization, *ISO 31000:2018 — Risk management — Guidelines*, ISO, Geneva, 2018.

[6] IEEE, *IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications*, 1998.

[7] IEEE, *IEEE Std 1633-2016 — Recommended Practice on Software Reliability*, 2016.

[8] IEEE / ISO / IEC, *ISO/IEC/IEEE 12207:2017 — Systems and software engineering — Software life cycle processes*, 2017.

[9] CMMI Institute, *CMMI for Development, Version 2.0*, ISACA, 2018.

[10] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7th ed., PMI, 2021.

[11] Congreso de la República de Colombia, *Ley Estatutaria 1581 de 2012 — Régimen General de Protección de Datos Personales*, 2012.